

Flight Booking System (Client–Server Architecture in C)

Awwab Ghole

April 30, 2026

1 Problem Statement

The objective of this project is to design and implement a **Flight Booking System** using a **client–server architecture** in a Linux environment.

The system supports:

- Multiple clients interacting concurrently with a server
- Admin functionalities: add, modify, delete flights
- User functionalities: search, book, cancel tickets
- Persistent storage using files
- Concurrency control and synchronization

2 System Architecture

The system consists of three main components:

2.1 Server

- Handles client requests
- Maintains database
- Uses multithreading for concurrency

2.2 Client

- CLI-based interface
- Sends structured requests
- Displays server responses

2.3 Database

- File-based storage:
 - users.dat
 - flights.dat
 - bookings.dat

3 Concepts and Implementation

3.1 Client–Server Communication

The system uses **UNIX domain sockets** for IPC.

```
sock = socket(AF_UNIX, SOCK_STREAM, 0);
connect(sock, ...);
```

Server side:

```
socket();
bind();
listen();
accept();
```

3.2 Structured Communication

Custom request-response protocol is used:

```
typedef struct {
    OpCode op;
    int session_user_id;
    int session_role;
} ClientRequest;
```

3.3 Multithreading

Each client is handled using a separate thread:

```
pthread_create(&tid, NULL, handle_client, client_sock);
```

3.4 File-Based Database

Binary files are used for storage:

```
open(DB_USERS, O_RDWR);
read(fd, &user, sizeof(User));
```

3.5 Concurrency Control

The system handles multiple clients concurrently using multithreading (pthread).

To ensure safe access to shared resources, the following synchronization mechanisms are used:

- **Mutex Locks:** pthread mutexes are used to enforce mutual exclusion when multiple threads attempt to modify shared data such as flight and booking records.
- **File Locking:** In addition to mutexes, fcntl-based file locks are used to ensure safe concurrent access at the file level, preventing race conditions across processes.

This hybrid approach ensures both thread-level and system-level synchronization.

3.6 Process Management (Logger)

A separate logger process is created using `fork()` and pipes:

```
pipe(log_pipe);  
fork();
```

This enables asynchronous logging.

3.7 Session Management

Session information is maintained per client:

```
int session_user_id;  
int session_role;
```

3.8 Role-Based Access Control

Access restrictions are enforced:

```
if (session_role != ROLE_ADMIN)
```

3.9 Booking Logic

Booking reduces available seats:

```
f.available_seats -= seats;
```

Cancellation restores seats:

```
f.available_seats += booked_seats;
```

4 Sample Output

4.1 Server Output

```
Server listening on /tmp/flight_booking.sock
New client connected.
User admin logged in.
Admin added flight AI101.
```

4.2 Client Menu

```
=== USER MENU ===

1. View Flights
2. Search Flights
3. Book Ticket
4. Cancel Ticket
5. View My Bookings
6. Logout
```

4.3 Flight Display

```
--- Available Flights ---
ID   Flight No   Origin   Dest    Total   Avail
1    AI101        Delhi    Mumbai  100     95
```

5 Challenges and Solutions

5.1 Concurrent File Access

Problem: Multiple clients accessing same file **Solution:** Used `fcntl()` locks

5.2 Data Consistency

Problem: Incorrect seat count **Solution:** Atomic updates with locking

5.3 Multi-client Handling

Problem: Blocking server **Solution:** Multithreading using `pthread`s

5.4 Efficient Logging

Problem: Logging slows down server **Solution:** Separate logger process

5.5 Session Handling

Problem: Stateless communication **Solution:** Maintain session variables

6 Conclusion

This project demonstrates a complete implementation of a client-server system using low-level C programming.

Key learnings include:

- Socket programming
- Concurrency and synchronization
- File handling as database
- Process and thread management

7 Future Enhancements

- Integration with MySQL
- GUI development (GTK / WinAPI)
- Secure authentication
- REST API version